2.2 Regular expressions

Vladimir Buskin
Catholic University of Eichstätt-Ingolstadt
ORCID: 0009-0005-5824-1012

Thomas Brunner
Catholic University of Eichstätt-Ingolstadt

Last updated: 14 October 2026, 19:59

This handout introduces regular expressions for advanced corpus queries in R and on corpus platforms, showing how to construct, refine and apply search patterns to linguistic data.

Table of contents

Preparation . . . 1
Recommended reading . . . 1
Regular expressions . . . 2
A regex cheat sheet . . . 5
Basic functions . . . 5
Character classes and alternatives . . . 5
Quantifiers . . . 5
Pre-defined character classes . . . 6
Querying POS-tagged corpora . . . 7
Exercises . . . 8
References . . . 9

Preparation

> Script
>
> You can find the full R script associated with this unit here.

Recommended reading

Lange and Leuckert (2020): Chapter 3.7

Detailed cheat sheet (DataCamp)

Regular expressions

Regular expressions (or 'regex') enable us to identify arbitrarily complex patterns in strings of text. Suppose we are interested in finding all inflectional forms of the lemma PROVIDE in a corpus, i.e., *provide, provides, providing* and *provided*. Instead of searching for all forms individually, we can construct a regular expression of the form

```
provid(e(s)?|ing|ed)
```

which can be read as

> 'Match the sequence of letters <provid>, followed by either <e>, <ing> or <ed>. The <e> may additionally be followed by an <s>.'

Notice how optionality is signified by the `?` operator and alternatives by `|`.

To use regular expressions in a `kwic()` query, first make sure you have loaded `quanteda`:

```
# Load library and corpus
library(quanteda)
ICE_GB <- readRDS("ICE_GB.RDS")
```

Now, the `valuetype` argument must be set to `"regex"`:

```
# Perform query
kwic_provide <- kwic(ICE_GB,
                     "provid(e(s)?|ing|ed)",
                     valuetype = "regex",
                     window = 20)
```

If we apply the `nrow()` function to `kwic_provide`, we will see that the data frame contains 419 observations. However, upon closer inspection, there appear to be a few false positives (*Provident*, *providential*, *provider* and *providers*):

```
table(kwic_provide$keyword)
```

```
     provide     provided      Provided     Provident providential      provider
         165          118             5             1             1             1
   providers     provides    providing     Providing
           3           72           52             1
```

! How does `kwic()` apply regular expressions?

`kwic()` checks the regular expression against **each token separately**, and a token counts as a match if the pattern occurs **anywhere** inside it. This is why *providential* and *provider* are included: They contain the sequence *provide*. For the same reason, the seemingly harmless query `kwic(ICE_GB, "eat", valuetype = "regex")` mostly returns words such as *great*, *heat*, *death* and *theatre*.
To match whole tokens only, we can **anchor** the pattern with `^` (beginning of the token) and `$` (end of the token), e.g., `"^eat$"`. Note also that `kwic()` ignores capitalisation by default.

There are two ways to handle this kind of output:

1. **Refine the search expression** further until you only match relevant instances.
2. Manually **sort out irrelevant cases** during qualitative annotation in spreadsheet software.

As a rule of thumb, you should consider improving your search expression if you obtain hundreds or even thousands of false hits. Should there be only a few false positives, it's usually easier to simply mark them as 'irrelevant' in your spreadsheet. A full workflow is demonstrated in the unit on data annotation.

Understanding true and false positives

When evaluating the accuracy of a regular expression (or a machine learning model such as a binary or multiclass classifier), we can use a **confusion matrix** to categorise the results:

	Actual Positive (should match)	**Actual Negative** (should not match)
Predicted Positive (pattern matches)	True Positive (TP)	False Positive (FP)
Predicted Negative (pattern does not match)	False Negative (FN)	True Negative (TN)

Using the current pattern `provid(e(s)?|ing|ed)`:

- **True Positives (TP)**: *provide, provides, providing, provided* (correctly matched relevant word forms)
- **False Positives (FP)**: *Provident, providential, provider, providers* (incorrectly matched irrelevant words)
- **False Negatives (FN)**: None in this case (we probably didn't miss any relevant cases)
- **True Negatives (TN)**: *proximity, product* etc. (correctly excluded irrelevant words)

Note that what counts as a true positive ultimately depends on how you define your linguistic variable. For instance, all five instances of capitalised *Provided* in the output above are actually the conjunction *provided (that)*, as in *Provided you keep everything else still*. If we are only interested in the verb PROVIDE, these are false positives, too. Two very common accuracy metrics are **precision** and **recall**. Precision answers the question: 'Of all matches, how many are correct?'.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

By contrast, recall tells us: 'Of all correct instances in the corpus, how many did we find?'.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

If we count all forms of *provide* as true positives, our query has a precision of $\frac{413}{413+6} \approx 0.986$.
The choice between the two metrics depends on which cell in the confusion matrix is considered more costly when predicted/matched incorrectly.

- High precision means a small proportion of false positives (e.g., a doctor who rarely diagnoses a disease in healthy patients, i.e., who raises few false alarms).
- Analogously, maximising recall corresponds to missing as few true positives as possible (e.g., ensuring that all true safety hazards on a construction site are detected, regardless of false alarms).

If both precision and recall are important, the **F1 score**, i.e., the harmonic mean of the two, offers a balanced measure:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A regex cheat sheet

Basic functions

Command	Definition	Example	Finds
`python`	The literal sequence of characters	`python`	*python*
`.`	Any character	`.ython`	*aython, bython…*
`^`	Beginning of the string (in `kwic()`: of the token)	`^pro`	*provide, product* (but not *approve*)
`$`	End of the string (in `kwic()`: of the token)	`ing$`	*walking, sing* (but not *singer*)
`\\`	Treats the following special character as a literal character	`etc\\.`	*etc.*

Character classes and alternatives

Command	Definition	Example	Finds
`[abc]`	Class of characters	`[jp]ython`	*jython, python*
`[^pP]`	Excluded class of characters	`[^pP]ython`	*jython, cython…*, but not *python* or *Python*
`(...|...)`	Alternatives linked by logical operator `or`	`P(ython|eter)`	*Python, Peter*

Quantifiers

Command	Definition	Example	Finds
`?`	Zero or one instance of the preceding character or group	`Py?thon`	*Python, Pthon*
`*`	Zero or more instances	`Py*thon`	*Python, Pthon, Pyyyython…*
		`P[Yy]*thon`	*Python, Pthon, PyYYython…*
`+`	One or more instances	`Py+thon`	*Python, Pyython, Pyyyython…*

Command	Definition	Example	Finds
`{1,3}`	Between *min* and *max* instances: `{min,max}`	`Py{1,3}thon`	*Python, Pyython, Pyyython*

Pre-defined character classes

> **Note**
>
> The double backslashes (`\\`) shown here are required whenever a regular expression is written inside an R string, including in `quanteda` functions: Since the backslash itself has a special meaning in R strings, it needs to be 'escaped' by a second one. In many other programming languages, a single backslash suffices (e.g., `\w`, `\d`, `\s`).

Command	Definition	Example	Finds
`\\w`	Any word character (letters, digits and the underscore _)	`\\w+ing`	*walking*, *running*, *42ing*
`\\W`	Any non-word character	`hello\\W+world`	*hello world*, *hello!!!world*
`\\d`	Any digit (0-9)	`\\d{3}-\\d{4}`	*555-1234*, *867-5309*
`\\D`	Any character that is not a digit	`\\D+`	*Hello!*, *Python_code*
`\\s`	Any whitespace character (space, tab, line break)	`word\\s+word`	*word word*
`\\b`	Word boundary	`\\bpython\\b`	Matches *python* as a whole word

> **Multi-word patterns in `kwic()`**
>
> Since `kwic()` checks each token separately and tokens never contain spaces, `\\s` and `\\W` can never match the gap between two words. Patterns such as `word\\s+word` will therefore not find anything in a `kwic()` query.
> Instead, write the patterns for the individual tokens separated by an **ordinary space** and wrap the entire expression in `phrase()` (see also 2.1 Concordancing). `phrase()` splits the expression at each space, so that every part is matched against one token:

```
# Finds 'eat' followed by 'a'
kwic(ICE_GB, phrase("^eat$ ^a$"), valuetype = "regex")
```

> Make sure to use exactly one space between the parts: Two consecutive spaces create an additional empty pattern, and the query will no longer find any matches.

docname	from	to	pre
S1A-009.txt	1198	1199	I_PRON must_AUX < , >
S1A-010.txt	958	959	to < , > actually_ADV
S1A-011.txt	3245	3246	: A > I_PRON have_AUX
S1A-011.txt	4159	4160	I_PRON ended_VERB up_ADP uhm_NOUN just_ADV
S1A-018.txt	455	456	order_VERB on_ADPe_NUM first_ADJ and_CCONJ_CCONJ then_ADV
S1A-019.txt	1038	1039	A > and_CCONJ everybody_PRON was_AUX

> The classes `\\s` and `\\W` do work, however, in R functions that operate on running text, such as `grepl()` or `gsub()`.

Querying POS-tagged corpora

The range of linguistic patterns to be matched can be extended further if the corpus contains additional linguistic annotation, such as the part of speech (POS) of each token. POS-tagged corpora open up the option of looking for more abstract patterns, such as all instances of the verb *eat* that are followed by a pronoun or noun. Load the POS-tagged corpus first:

```
# Load library and corpus
ICE_GB_POS <- readRDS("ICE_GB_POS.RDS")
```

In this corpus, every token has the form `word_TAG` (e.g., `eat_VERB`). Next, we provide a regular expression that captures all inflectional forms of *eat* tagged as verbs, followed by any token bearing the tag `_PRON` or `_NOUN`. Since the underscore counts as a word character, `\\b` marks the boundaries of the entire token here: `eat\\b` would not match `eat_VERB`, but `eat_VERB\\b` does.

```
# Perform query
kwic_eat_POS <- kwic(ICE_GB_POS,
                     phrase("\\b(ate|eat(s|ing|en)?)_VERB\\b
                      ↪  _(PRON|NOUN)"),
                     valuetype = "regex",
                     window = 5)

head(kwic_eat_POS)
```

One R package that supplies functions for tokenisation, POS tagging and even dependency parsing for dozens of languages is `udpipe`. Its models rely on a common set of tags developed by the **Universal Dependencies** project, which are listed here:

Universal Dependencies: POS tagset

Cf. https://universaldependencies.org/u/pos/.

POS Tag	Description
ADJ	Adjective: describes a noun (e.g., *big*, *old*, *green*, *first*)
ADP	Adposition: prepositions and postpositions (e.g., *in*, *to*, *over*)
ADV	Adverb: modifies verbs, adjectives, or other adverbs (e.g., *quickly*, *very*)
AUX	Auxiliary: helps form verb tenses, moods, or voices (e.g., *is*, *have*, *will*)
CCONJ	Coordinating conjunction: links words, phrases, or clauses (e.g., *and*, *or*, *but*)
DET	Determiner: introduces nouns (e.g., *the*, *a*, *some*, *my*)
INTJ	Interjection: expresses emotion or reaction (e.g., *oh*, *wow*, *hello*)
NOUN	Noun: person, place, thing, or concept (e.g., *cat*, *city*, *idea*)
NUM	Numeral: expresses a number or ranking (e.g., *one*, *two*, *second*)
PART	Particle: adds meaning without being an independent word class (e.g., *not*, *to* as in *to run*)
PRON	Pronoun: replaces nouns (e.g., *he*, *she*, *they*, *it*)
PROPN	Proper noun: names specific entities (e.g., *London*, *John*)
PUNCT	Punctuation: marks boundaries in text (. , ! ?)
SCONJ	Subordinating conjunction: links clauses, often indicating dependency (e.g., *if*, *because*, *although*)
SYM	Symbol: non-alphanumeric symbol (e.g., %, &, #)
VERB	Verb: action or state (e.g., *run*, *be*, *have*)
X	Other: used when a word doesn't fit into other categories

Exercises

Solutions

You can find the solutions to the exercises here.

Exercise 1. How could you refine the search expression for PROVIDE `"provid(e(s)?|ing|ed)"` to get rid of the irrelevant cases?

Exercise 2. Write elegant regular expressions that find all inflectional forms of the following verbs:

- *accept*

- *attach*
- *swim*
- *know*
- *forget*

Exercise 3. Using the POS-tagged corpus, find all nouns ending in *-er*.

Exercise 4. Find all four-digit numbers.

Exercise 5. Using the POS-tagged corpus, find all verbs that are followed by a preposition.

References

Lange, Claudia, and Sven Leuckert. 2020. *Corpus Linguistics for World Englishes: A Guide for Research*. New York: Taylor; Francis.